Understood—here is the **same case study redesigned as a pure conceptual + implementation assignment**, with **no code**, so students must build everything themselves.

📘 Case Study: User Management System (ASP.NET Core MVC)

🎯 Objective

Design and implement a **User Management Module** using ASP.NET Core MVC where users can:

- Register
- Login
- View Profile
- Edit Profile

🧩 1. Business Scenario

A small organization needs a web application to manage users:

- New users should be able to **create an account**
- Existing users should be able to **log in**
- Logged-in users should be able to **view and update their profile**
- Unauthorized users should not access profile pages

---

2. Project Architecture (Layered MVC)

```
Solution
 ├── Controllers
 │     └── UserController.cs
 ├── Models
 │     └── User.cs
 ├── ViewModels
 │     └── UserViewModel.cs
 ├── Views
 │     └── User
 │          ├── Register.cshtml
 │          ├── Login.cshtml
 │          ├── Profile.cshtml
 │          └── Edit.cshtml
 └── wwwroot
```

Students must create a project using:

- MVC Pattern
- Separate folders for:
  - Controllers
  - Models
  - Views
  - ViewModels

👉 Expected outcome:

- Clear separation of concerns
- Proper naming conventions

🧱 3. Model Design (Student Task)

Create a **User Model** with appropriate properties such as:

- Id
- Name
- Email
- Password
- Confirm Password

📌 **Requirements:**

- Apply **data validation rules**
- Ensure:
  - Name is mandatory
  - Email is valid
  - Password has minimum length
  - Password and Confirm Password match

✅ 4. Model Validation

Students must:

- Use **Data Annotations**
- Display validation errors in UI

📌 **Expected Learning:**

- Server-side validation
- Client-side validation using Razor helpers

🔄 5. ViewModel Design

Students should create a **ViewModel** for:

- Displaying user profile

📌 **Requirement:**

- Do NOT expose sensitive data (like password)
- Include only required fields for UI

🎮 6. Controller Design

Create a **User Controller** with the following actions:

📌 **Required Actions:**

1. Register (GET)
2. Register (POST)
3. Login (GET)
4. Login (POST)
5. Profile (GET)
6. Edit Profile (GET & POST)

📌 **Responsibilities:**

- Handle user input
- Validate model
- Redirect between pages
- Manage session

🌐 7. Routing

Students must:

- Configure **default routing**
- Ensure application starts with Login page

📌 **Example URLs (to be achieved):**

- /User/Register
- /User/Login
- /User/Profile

🔐 8. Session Management

Students should implement session to:

- Store logged-in user information
- Restrict unauthorized access

📌 **Requirements:**

- Store user identity after login
- Retrieve session data in Profile
- Redirect to Login if session is missing

🖼️ 9. Views (UI Design)

Students must create Razor Views for:

- Register Page

- Login Page
- Profile Page
- Edit Profile Page

📌 **Requirements:**

- Use **Tag Helpers / HTML Helpers**
- Display validation messages
- Maintain clean UI structure

🔄 10. Application Flow

Students should implement the following flow:

1. User opens Register page
2. Submits details → validation occurs
3. On success → redirect to Login
4. User logs in → session created
5. Redirect to Profile page
6. Profile displays user data via ViewModel
7. User can edit profile
8. Logout should clear session

🧠 11. Key Concepts Covered

This case study ensures practical understanding of:

- MVC Architecture (Model, View, Controller)
- Model Validation
- Routing
- Session Management
- ViewModels

- Razor Views

📝 12. Student Implementation Tasks

🔹 Basic Tasks

1. Create User Model with validations
2. Implement User Controller with all actions
3. Design all required Views
4. Configure routing properly

🔹 Intermediate Tasks

5. Implement session-based login
6. Restrict Profile access without login
7. Use ViewModel for Profile page

🔹 Advanced Tasks

8. Add Logout functionality
9. Add Edit Profile feature